NUST CHIP DESIGN CENTRE

C / C++ Programming

Lab # 10
Finite State Machine

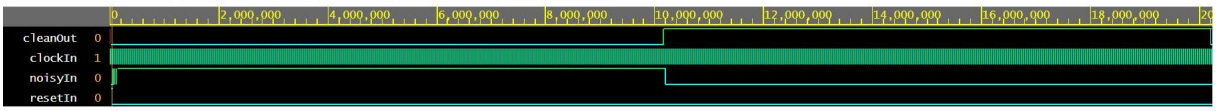
Name	*Waleed Amir*
Instructor	*Musadiq Hussain*
Date	*7th August 2025*

1. <u>In-Lab Tasks</u>: (*Write your lab task & screenshots here*)

i. Task 1: Debouncer Module

The debouncer module is important for cleaning up noisy input signals from physical Switches due to switch bouncing. It makes sure that a single press or release is registered as a stable signal, preventing false triggers due to mechanical bounce.

Waveform:

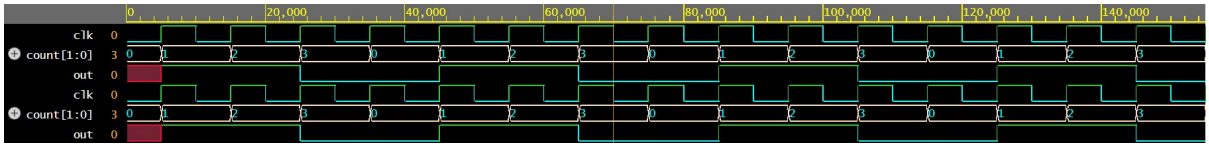


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

ii. Task 2: Clock Divider Module

The clkDiv module is responsible for generating slower clock signals from a high-frequency input clock. Since the system is required to work at 25Mhz but the FPGA freq is 100Mhz, we need to down sample the provided clock.

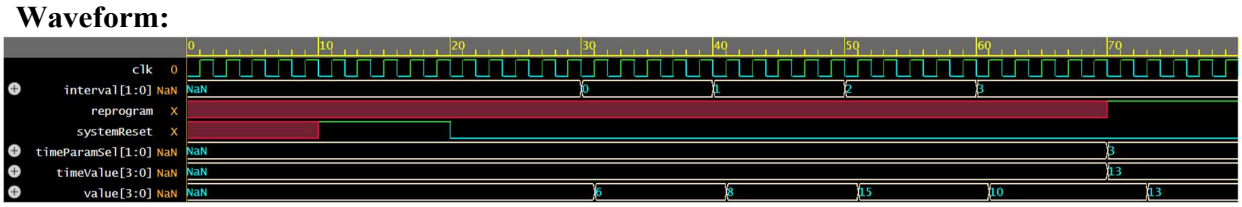
Waveform:



iii. Task 3: Time Parameter Module

The timeParams module stores and provides the various time delays required by the anti-theft system, such as arming delay, driver door trigger delay, passenger doortrigger delay, and alarm duration. It also allows for reprogramming these values.

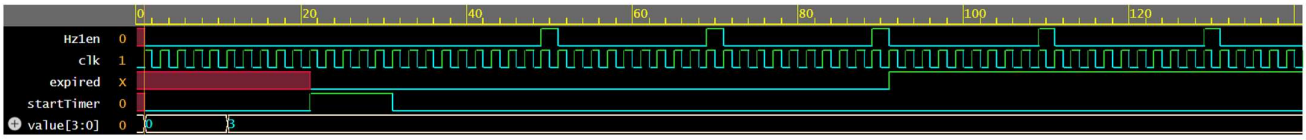
Waveform:



iv. Task 4: Timer Module

The module timer works in conjunction with the module timeParams to provide countdown functionality. It takes a time value and a start signal, then asserts an expired signal once the countdown is complete. It also generates a 1Hz enable signal.

Waveform:

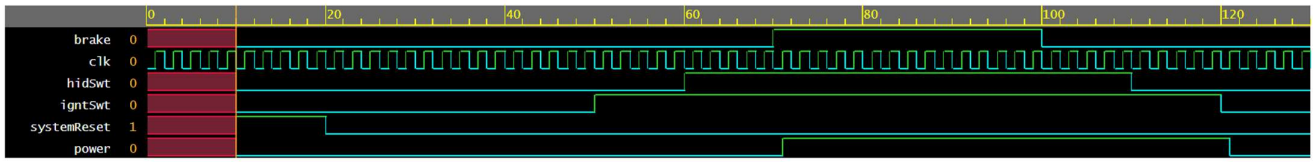


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

v. Task 5: Fuel Pump Logic Module

The fuelPumpLogic module controls the vehicle's fuel pump. It's designed to cut off fuel supply when the anti-theft system is active and triggered, and to re-enable it only when correct disarming sequence (hidden switch and brake pedal) is activated.

Waveform:

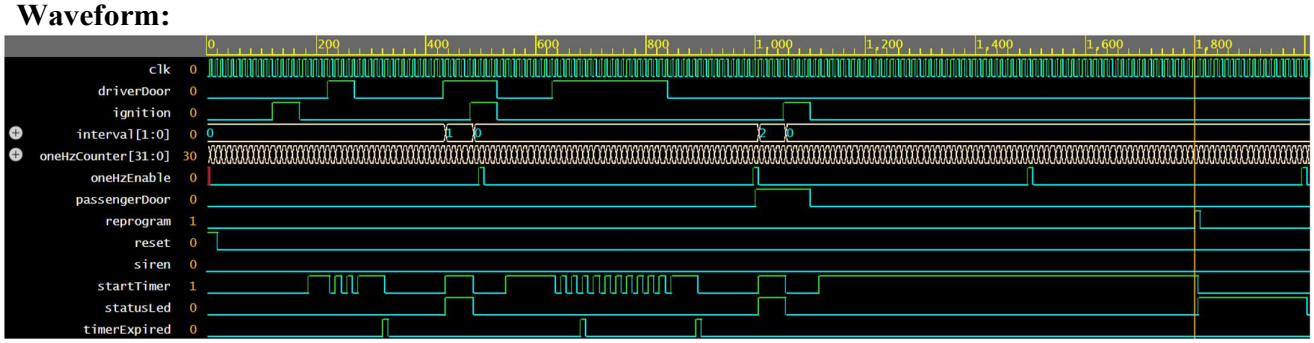


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

vi. Task 7: Anti-theft FSM Module

The antiTheftFSM module is the brain of the system, controlling the overall behavior based on inputs from various sensors and managing the arming, triggering, and alarming sequences. It orche strates the interaction between all other modules.

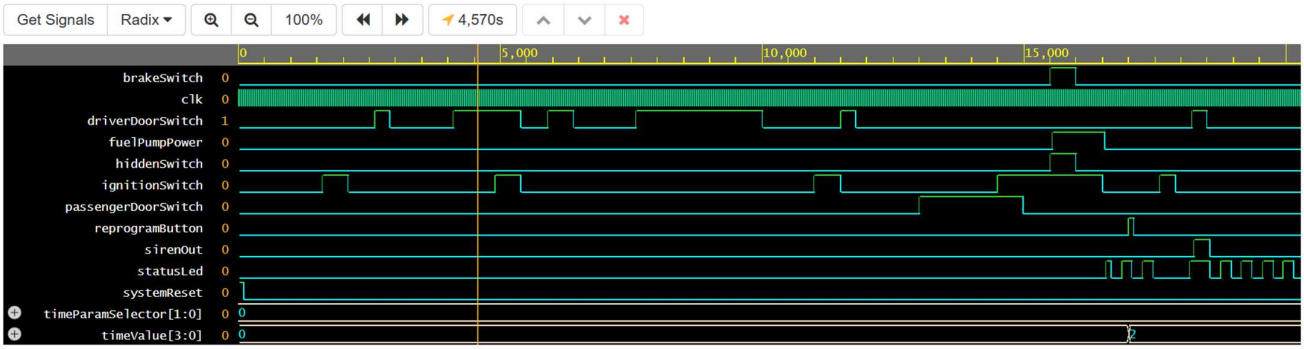
Waveform:



vii. Task 8: Top-Level Module

The topLevel module integrates all the individual modules to form the complete anti-theft system.

Waveform:



1. What could happen if an input were not synchronized to the clock?

Metastability: When a asynchronous input changes close to the clock edge,the receiving may enter a “metastable state” where its output isneither a valid logic 0 nor 1, but rather X.

Setup and Hold Time Violations: Asynchronous inputs may violate the setup and hold time requirements by taking to less or too much time before or after the clock, this leads to unreliable data capture.

Race Conditions: Different parts of the circuit may see the input change at different times or even changes simultaneously which causes confusion as to which signal to read first, this leads to inconsistent behavior like entering a wrong state.

Glitches and False Triggers: Mechanical switches can bounce, creating multipletransitions that could be read as separate events they are not debounced as well as synchronized.

2. Describe your fuel pump and siren sound generator logic.

Fuel Pump Logic:

The fuel pump logic is implemented as a simple 3-state moore style FSM:

- **OFF**: This is the default state when ignition is off and the fuel pump power is disabled.
- **WAIT**: Entered when ignition turns on. This state waits for both hiddenSwitch and brakePedal to be pressed simultaneously.
- **ON**: The system enters this state when the secret sequence (hiddenSwitch + brakePedal) is detected. Fuel pump power is enabled and latched until ignition is turned off.

Siren Generator Logic:

- **Tone Selection**: Alternates between 440Hz and 880Hz every 2 seconds using acounter.
- **Frequency Generation**: Uses separate counters for each frequency to generatesquare waves:
  - 440Hz: 25MHz / (2 × 440) = 28,409 cycles per half-period880Hz:
  - 25MHz / (2 × 880) = 14,204 cycles per half-period
- **Output Control**: Only generates audio when sirenOn is asserted, otherwise it outputs silence.

3. Describe your divider module.

Firstly, there is an outer clock dividier module that converts the FPGA's 100MHz clock frequency to 25 Mhz which is required for the overall system. The second divider module is built within the timer module and converts the25MHz system clock into a 1Hz enablesignal:

- **Counter Size**: Uses a 25-bit counter to count up to 25,000,000 cycles (1 second at25MHz).
- **Reset Behavior**: The divider counter resets to 0 when startTimer is asserted,ensuring the first 1Hz pulse occurs exactly 1 second after the timer starts.
- **Output**: Generates oneHzEnable pulse for exactly one clock cycle when thecounter reaches its maximum value since the clock is read at its posedge, only one pulse needs enough.
- **Usage**: The oneHzEnable signal' posedge is used by the timer countdown logic and therefore only requires on pulse when the timer reaches its rollover value.

4. Describe the design flow for your system.

1. **FSM Design**:

- Designed state transitions based on sensor inputs and timer expiration
- Implemented proper output logic for each state

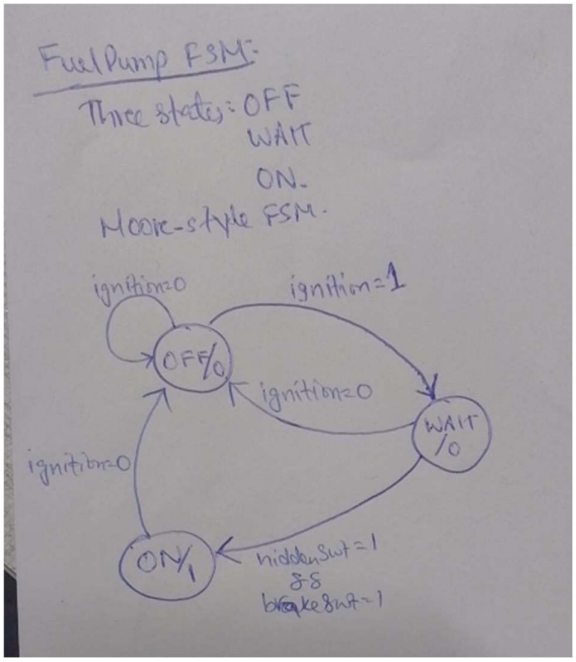
2. **Integration**:

- Created a top-level module to instantiate and connect all sub-modules
- Ensured proper signal routing between modules
- Added debouncing for all external inputs

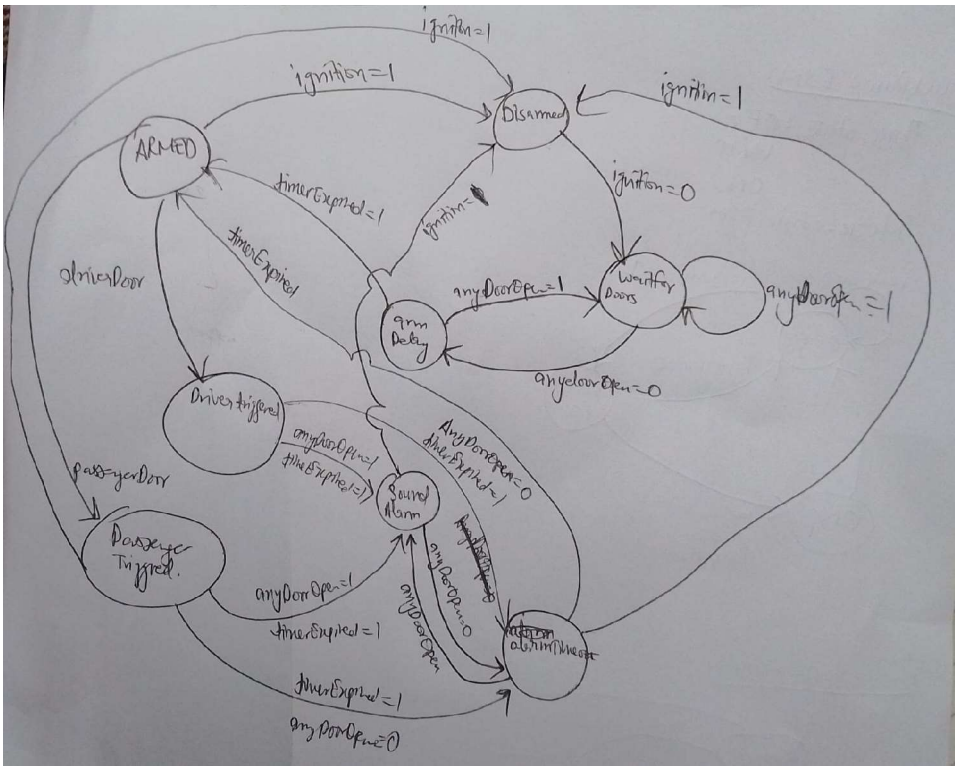
3. **Verification**:

- Created testbenches for individual modules
- Developed comprehensive system-level testbench
- Verified functionality through simulation

FSM diagrams:
For fuel logic:



For anti theft FSM module:


ignition=1
ignition=1
ignition=1
ARMED
Disarmed
timerExpired=1
ignition=0
ignition=
timerExpired
driverDoor
anyDoorOpen=1
Wait for Doors
anyDoorOpen=1
Arm Delay
anyDoorOpen=0
Driver triggered
anyDoorOpen=1
timerExpired=1
AnyDoorOpen=0
timerExpired=1
passengerDoor
Sound Alarm
Passenger Triggered
anyDoorOpen=1
anyDoorOpen=0
anyDoorOpen
timerExpired=1
timerExpired=1
anyDoorOpen=0